

1. Write a R program to take input from the user (name and age) and display the values. Also print the version of R installation

R Programming Code :

```
# Read name from user input
name <- readline(prompt = "Input your name: ")
# Read age from user input
age <- readline(prompt = "Input your age: ")
# Print the formatted output
print(paste("My name is", name, "and I am", age, "years old."))
# Print the R version string
print(R.version.string)
O/P
```

```
> name <- readline(prompt = "Input your name: ")
Input your name: Trefor Renat
> age <- readline(prompt = "Input your age: ")
Input your age: 25
> # Print the formatted output
> print(paste("My name is", name, "and I am", age, "years old."))
[1] "My name is Trefor Renat and I am 25 years old."
> print(R.version.string)
[1] "R version 4.4.1 (2024-06-14 ucrt)"
```

2) Write a R program to create a sequence of numbers from 20 to 50 and find the mean of numbers from 20 to 60 and sum of numbers from 51 to 91.

R Programming Code :

```
# Print a message indicating that the sequence of numbers from 20 to 50 will be shown
print("Sequence of numbers from 20 to 50:")
# Generate and print a sequence of numbers from 20 to 50
print(seq(20,50))

# Print a message indicating that the mean of numbers from 20 to 60 will be shown
print("Mean of numbers from 20 to 60:")
# Calculate and print the mean of numbers from 20 to 60
print(mean(20:60))
# Print a message indicating that the sum of numbers from 51 to 91 will be shown
```

```
print("Sum of numbers from 51 to 91:")  
# Calculate and print the sum of numbers from 51 to 91  
print(sum(51:91))
```

Output:

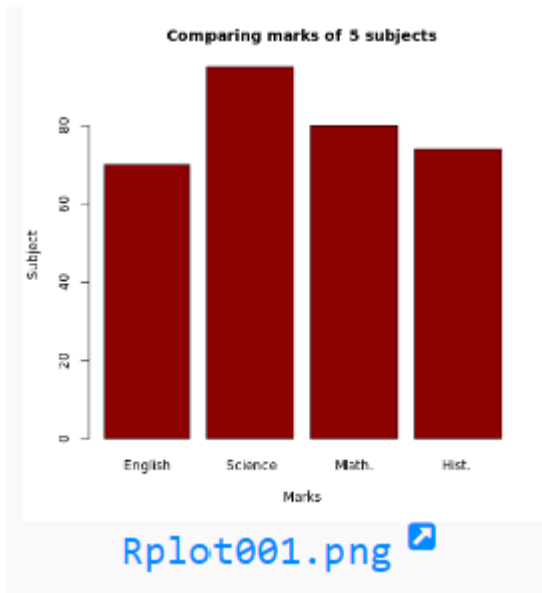
```
[1] "Sequence of numbers from 20 to 50:"  
[1] 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38  
39 40 41 42 43 44  
[26] 45 46 47 48 49 50  
[1] "Mean of numbers from 20 to 60:"  
[1] 40  
[1] "Sum of numbers from 51 to 91:"  
[1] 2911
```

3) Write a R program to create a simple bar plot of five subjects marks.

R Programming Code :

```
# Define a vector of marks for the 4 subjects  
marks = c(70, 95, 80, 74)  
# Create a bar plot with the specified marks  
barplot(marks,  
  # Set the title of the bar plot  
  main = "Comparing marks of 5 subjects",  
  # Label the x-axis as "Marks"  
  xlab = "Marks",  
  # Label the y-axis as "Subject"  
  ylab = "Subject",  
  # Set the names of the bars  
  names.arg = c("English", "Science", "Math.", "Hist."),  
  # Set the color of the bars to dark red  
  col = "darkred",  
  # Ensure the bars are vertical  
  horiz = FALSE)
```

Output:



4) Write a R program to get the unique elements of a given string and unique numbers of vector.

R Programming Code :

```
# Assign a string to the variable str1
```

```
str1 = "The quick brown fox jumps over the lazy dog."
```

```
# Print a message indicating that the following output is the original string
```

```
print("Original vector(string)")
```

```
# Print the content of the string
```

```
print(str1)
```

```
# Print a message indicating that the following output will show unique elements of the string
```

```
print("Unique elements of the said vector:")
```

```
# Convert the string to lowercase and print unique characters
```

```
print(unique(tolower(str1)))
```

```
# Create a vector of numbers with some duplicates
```

```
nums = c(1, 2, 2, 3, 4, 4, 5, 6)
```

```
# Print a message indicating that the following output is the original number vector
```

```
print("Original vector(number)")
```

```
# Print the content of the number vector
```

```
print(nums)
```

```
# Print a message indicating that the following output will show unique elements of the number vector
```

```
print("Unique elements of the said vector:")
```

```
# Print unique numbers from the vector
```

```
print(unique(nums))
```

Output:

```
[1] "Original vector(string)"
[1] "The quick brown fox jumps over the lazy dog."
[1] "Unique elements of the said vector:"
[1] "the quick brown fox jumps over the lazy dog."
[1] "Original vector(number)"
[1] 1 2 2 3 4 4 5 6
[1] "Unique elements of the said vector:"
[1] 1 2 3 4 5 6
```

7) Write a R program to create a list containing a vector, a matrix and a list and give names to the elements in the list. Access the first and second element of the list.

Programming

R Programming Code :

```
l# Create a list with a vector, a matrix, and another list
list_data <- list(c("Red","Green","Black"), matrix(c(1,3,5,7,9,11), nrow = 2),
list("Python", "PHP", "Java"))
```

```
# Print the original list
```

```
print("List:")
print(list_data)
```

```
# Assign names to each element of the list
```

```
names(list_data) = c("Color", "Odd numbers", "Language(s)")
```

```
# Print the list with the assigned names
```

```
print("List with column names:")
print(list_data)
```

```
# Access and print the first element of the list (the vector)
```

```
print('1st element:')
print(list_data[1])
```

```
# Access and print the second element of the list (the matrix)
```

```
print('2nd element:')
print(list_data[2])
```

Output:

```
[1] "List:"
[[1]]
[1] "Red" "Green" "Black"

[[2]]
[,1] [,2] [,3]
```

```
[1,] 1 5 9
[2,] 3 7 11
```

```
[[3]]
[[3]][[1]]
[1] "Python"
```

```
[[3]][[2]]
[1] "PHP"
```

```
[[3]][[3]]
[1] "Java"
```

```
[1] "List with column names:"
$Color
[1] "Red" "Green" "Black"
```

```
$`Odd numbers`
[,1] [,2] [,3]
[1,] 1 5 9
[2,] 3 7 11
```

```
$`Language(s)`
$`Language(s)`[[1]]
[1] "Python"
```

```
$`Language(s)`[[2]]
[1] "PHP"
```

```
$`Language(s)`[[3]]
[1] "Java"
```

```
[1] "1st element:"
$Color
[1] "Red" "Green" "Black"
```

```
[1] "2nd element:"
$`Odd numbers`
[,1] [,2] [,3]
[1,] 1 5 9
[2,] 3 7 11
```

13) Write a R program to create a data frame using two given vectors and display the duplicated elements and unique rows of the said data frame.

Programming

Sample Solution :

R Programming Code :

```
# Create vector 'a' with specified values
```

```
a = c(10, 20, 10, 10, 40, 50, 20, 30)
```

```
# Create vector 'b' with specified values
```

```
b = c(10, 30, 10, 20, 0, 50, 30, 30)
```

```
# Print a message indicating that the original data frame will be displayed
```

```
print("Original data frame:")
```

```
# Combine vectors 'a' and 'b' into a data frame 'ab'
```

```
ab = data.frame(a, b)
```

```
# Print the data frame 'ab'
```

```
print(ab)
```

```
# Print a message indicating that duplicated elements in the data frame will be shown
```

```
print("Duplicate elements of the said data frame:")
```

```
# Display duplicated elements of the data frame 'ab'
```

```
print(duplicated(ab))
```

```
# Print a message indicating that unique rows of the data frame will be shown
```

```
print("Unique rows of the said data frame:")
```

```
# Display unique rows of the data frame 'ab'
```

```
print(unique(ab))
```

Output:

```
[1] "Original data frame:"
```

```
  a b
1 10 10
2 20 30
3 10 10
4 10 20
5 40  0
6 50 50
7 20 30
8 30 30
```

```
[1] "Duplicate elements of the said data frame:"
```

```
[1] FALSE FALSE TRUE FALSE FALSE FALSE TRUE FALSE
```

```
[1] "Unique rows of the said data frame:"
```

```
  a b  
1 10 10  
2 20 30  
4 10 20  
5 40  0  
6 50 50  
8 30 30
```

18) Write a R program to create two 2x3 matrix and add, subtract, multiply and divide the matrixes.

Sample Solution:

R Programming Code:

```
# Create the first 2x3 matrix with elements 1 to 6
```

```
m1 = matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)
```

```
# Print a message indicating the following output is Matrix-1
```

```
print("Matrix-1:")
```

```
# Print the contents of the first matrix
```

```
print(m1)
```

```
# Create the second 2x3 matrix with elements 0, 1, 2, 3, 0, 2
```

```
m2 = matrix(c(0, 1, 2, 3, 0, 2), nrow = 2)
```

```
# Print a message indicating the following output is Matrix-2
```

```
print("Matrix-2:")
```

```
# Print the contents of the second matrix
```

```
print(m2)
```

```
# Perform element-wise addition of the two matrices
```

```
result = m1 + m2
```

```
# Print a message indicating the following output is the result of addition
```

```
print("Result of addition")
```

```
# Print the result of the matrix addition
```

```
print(result)
```

```
# Perform element-wise subtraction of the two matrices
```

```
result = m1 - m2
```

```
# Print a message indicating the following output is the result of subtraction
```

```
print("Result of subtraction")
```

```
# Print the result of the matrix subtraction
```

```
print(result)
```

```

# Perform element-wise multiplication of the two matrices
result = m1 * m2

# Print a message indicating the following output is the result of multiplication
print("Result of multiplication")

# Print the result of the matrix multiplication
print(result)

# Perform element-wise division of the two matrices
result = m1 / m2

# Print a message indicating the following output is the result of division
print("Result of division:")

# Print the result of the matrix division
print(result)
Output:
[1] "Matrix-1:"
      [,1] [,2] [,3]
[1,]   1   3   5
[2,]   2   4   6
[1] "Matrix-2:"
      [,1] [,2] [,3]
[1,]   0   2   0
[2,]   1   3   2
[1] "Result of addition"
      [,1] [,2] [,3]
[1,]   1   5   5
[2,]   3   7   8
[1] "Result of subtraction"
      [,1] [,2] [,3]
[1,]   1   1   5
[2,]   1   1   4
[1] "Result of multiplication"
      [,1] [,2] [,3]
[1,]   0   6   0
[2,]   2  12  12
[1] "Result of division:"
      [,1] [,2] [,3]
[1,] Inf 1.500000 Inf
[2,]  2 1.333333  3

19) Using R import the data from Excel/.CSV file and find mean, median, mode, quartiles.
# Load data
data <- read.csv("data.csv")

```



```
# Select column
x <- data$marks
```

```
# Mean
mean(x, na.rm = TRUE)
```

```
# Median
median(x, na.rm = TRUE)
```

```
# Mode
mode_func <- function(x) {
  uniq_x <- unique(x)
  uniq_x[which.max(tabulate(match(x, uniq_x)))]
}
mode_func(x)
```

```
# Quartiles
quantile(x, na.rm = TRUE)
```

23) Write a R program to compare two data frames to find the row(s) in first data frame that are not present in second data frame.

Sample Solution:

R Programming Code:

```
# Create the first data frame with sales data for three items
df_90 = data.frame(
  "item" = c("item1", "item2", "item3"),    # Column for item names
  "Jan_sale" = c(12, 14, 12),               # Sales in January
  "Feb_sale" = c(11, 12, 15),              # Sales in February
  "Mar_sale" = c(12, 14, 15)               # Sales in March
)

# Create the second data frame with sales data for three items
df_91 = data.frame(
  "item" = c("item1", "item2", "item3"),    # Column for item names
  "Jan_sale" = c(12, 14, 12),               # Sales in January
  "Feb_sale" = c(11, 12, 15),              # Sales in February
  "Mar_sale" = c(12, 15, 18)               # Sales in March
)

# Print a message indicating the following output is the original dataframes
print("Original Dataframes:")

# Print the first data frame
print(df_90)

# Print the second data frame
print(df_91)
```

```
# Print a message indicating the following output is the rows in the first data frame not present in the second data frame
```

```
print("Row(s) in first data frame that are not present in second data frame:")
```

```
# Find and print rows in the first data frame that are not present in the second data frame
```

```
print(setdiff(df_90, df_91))
```

Output:

```
[1] "Original Dataframes:"
```

```
  item Jan_sale Feb_sale Mar_sale
```

```
1 item1      12      11      12
```

```
2 item2      14      12      14
```

```
3 item3      12      15      15
```

```
  item Jan_sale Feb_sale Mar_sale
```

```
1 item1      12      11      12
```

```
2 item2      14      12      15
```

```
3 item3      12      15      18
```

```
[1] "Row(s) in first data frame that are not present in second data frame:"
```

```
$Mar_sale
```

```
[1] 12 14 15
```

26) Write a R program to sort a Vector in ascending and descending order.

Programming

Sample Solution :

R Programming Code :

```
# Create a numeric vector 'x' with elements 10, 20, 30, 25, 9, and 26
```

```
x = c(10, 20, 30, 25, 9, 26)
```

```
# Print a message indicating the display of the original vector
```

```
print("Original Vectors:")
```

```
# Print the contents of vector 'x'
```

```
print(x)
```

```
# Print a message indicating sorting in ascending order
```

```
print("Sort in ascending order:")
```

```
# Sort vector 'x' in ascending order and print the result
```

```
print(sort(x))
```

```
# Print a message indicating sorting in descending order
```

```
print("Sort in descending order:")
```

```
# Sort vector 'x' in descending order and print the result
```

```
print(sort(x, decreasing=TRUE))
```

Output:

```
[1] "Original Vectors:"
```

```
[1] 10 20 30 25 9 26
```

```
[1] "Sort in ascending order:"
```

```
[1] 9 10 20 25 26 30
[1] "Sort in descending order:"
[1] 30 26 25 20 10 9
```

27) Write a R program to extract first 10 english letter in lower case and last 10 letters in upper case and extract letters between 22nd to 24th letters in upper case.

Books & Literature

Note: Use built-in datasets letters and LETTERS.

Sample Solution :

R Programming Code :

```
# Print a message indicating the first 10 letters in lower case
```

```
print("First 10 letters in lower case:")
```

```
# Get the first 10 letters of the alphabet in lower case
```

```
t = head(letters, 10)
```

```
# Print the first 10 letters in lower case
```

```
print(t)
```

```
# Print a message indicating the last 10 letters in upper case
```

```
print("Last 10 letters in upper case:")
```

```
# Get the last 10 letters of the alphabet in upper case
```

```
t = tail(LETTERS, 10)
```

```
# Print the last 10 letters in upper case
```

```
print(t)
```

```
# Print a message indicating the letters between the 22nd and 24th in upper case
```

```
print("Letters between 22nd to 24th letters in upper case:")
```

```
# Get the letters between the 22nd and 24th positions in upper case
```

```
e = tail(LETTERS[22:24])
```

```
# Print the letters between the 22nd and 24th positions in upper case
```

```
print(e)
```

Output:

```
[1] "First 10 letters in lower case:"
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j"
```

```
[1] "Last 10 letters in upper case:"
```

```
[1] "Q" "R" "S" "T" "U" "V" "W" "X" "Y" "Z"
```

```
[1] "Letters between 22nd to 24th letters in upper case:"
```

```
[1] "V" "W" "X"
```

32) `weather <- read.csv("weather.csv")`

```
# Column selection
```

```
weather[,1]
```

```
weather[, c(2,3)]
```

```
weather$Temperature
```

```
weather[, c("Temperature", "Wind")]
```

```
# Scatter plot
```

```
plot(weather$Wind, weather$Temperature,  
      main="Wind vs Temperature",  
      xlab="Wind Speed",  
      ylab="Temperature",  
      pch=19)
```

20) Using R import the data from Excel/.CSV file and find standard deviation, variance and co-variance.

```
# Load data
```

```
data <- read.csv("weather.csv")
```

```
# Select columns
```

```
temp <- data$Temperature
```

```
wind <- data$Wind
```

```
# Standard Deviation
```

```
sd(temp, na.rm = TRUE)
```

```
sd(wind, na.rm = TRUE)
```

```
# Variance
```

```
var(temp, na.rm = TRUE)
```

```
var(wind, na.rm = TRUE)
```

```
# Covariance
```

```
cov(temp, wind, use = "complete.obs")
```

9) Write a R program to merge two given lists into one list.

Sample Solution :

R Programming Code :

```
# Create the first list containing numbers
```

```
n1 = list(1,2,3)
```

```
# Create the second list containing color names
```

```
c1 = list("Red", "Green", "Black")
```

```
# Print a message indicating the following output will be the original lists
```

```
print("Original lists:")
```

```
# Print the first list
```

```
print(n1)
```

```
# Print the second list
```

```
print(c1)
```

```
# Print a message indicating that the lists will be merged
```

```
print("Merge the said lists:")
```

```
# Merge the two lists into one
```

```
m1ist = c(n1, c1)
```

```
# Print a message indicating the following output will be the new merged list
```

```
print("New merged list:")
```

```
# Print the merged list
```

```
print(m1ist)
```

```
Output:
```

```
[1] "Original lists:"
```

```
[[1]]
```

```
[1] 1
```

```
[[2]]
```

```
[1] 2
```

```
[[3]]
```

```
[1] 3
```

```
[[1]]
```

```
[1] "Red"
```

```
[[2]]
```

```
[1] "Green"
```

```
[[3]]
```

```
[1] "Black"
```

```
[1] "Merge the said lists:"
```

```
[1] "New merged list:"
```

```
[[1]]
```

```
[1] 1
```

```
[[2]]
```

```
[1] 2
```

```
[[3]]
```

```
[1] 3
```

```
[[4]]
```

```
[1] "Red"
```

[[5]]

[1] "Green"

[[6]]

[1] "Black"